

TEMA 1 · MONITOREO DEL MOTOR



ANÁLISIS DE RENDIMIENTO CON SQL PROFILER Y EXTENDED EVENTS

Captura y analiza en tiempo real los eventos del motor SQL Server (consultas, bloqueos, uso de CPU y E/S) para detectar cuellos de botella antes de que afecten a los usuarios de GradosTítulos.

SQL PROFILER

01

Herramienta gráfica clásica que captura eventos como **RPC:Completed** y **SQL:BatchCompleted** en una traza. Es intuitiva, pero genera mayor sobrecarga sobre el servidor.

EXTENDED EVENTS (XE)

02

Motor de diagnóstico ligero que reemplaza a Profiler. Permite crear sesiones filtradas, exportarlas a archivos **.xel** y consultarlas directamente con T-SQL.

MÉTRICAS CLAVE

03

Duración, lecturas lógicas, CPU, escrituras y tipos de espera (**waits**) son las métricas que revelan qué consulta está degradando el rendimiento.

► SCRIPT T-SQL DE REFERENCIA

```
CREATE EVENT SESSION MonitoreoGT ON SERVER
ADD EVENT sqlserver.sql_statement_completed(
    ACTION(sqlserver.sql_text, sqlserver.database_name))
ADD TARGET package0.event_file(SET filename=monitoreo.xel);
GO
ALTER EVENT SESSION MonitoreoGT ON SERVER STATE = START;
```

SOBRECARGA (OVERHEAD) SOBRE EL SERVIDOR

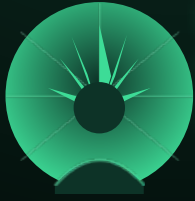
PROFILER > EXTENDED EVENTS EN LA MISMA CARGA DE TRABAJO

45%

BUENAS PRÁCTICAS

- ✓ Preferir **Extended Events** sobre Profiler en ambientes de producción.
- ✓ Filtrar sesiones por duración mínima para capturar solo lo relevante.
- ✓ Limitar el tamaño máximo del archivo de eventos (**max_file_size**).
- ✓ Detener y eliminar sesiones que ya no se estén usando.

TEMA 2 ESTRUCTURAS DE ACCESO



ESTADÍSTICAS E ÍNDICES: CREACIÓN, FRAGMENTACIÓN Y MANTENIMIENTO

Los índices aceleran la búsqueda de datos y las estadísticas le indican al optimizador cuántas filas esperar. Ambos deben mantenerse para que el motor elija siempre el mejor plan de ejecución.

ÍNDICES CLUSTERED / NONCLUSTERED

01

El **clustered** ordena físicamente los datos de la tabla; los **nonclustered** crean estructuras aparte con punteros hacia los registros.

FRAGMENTACIÓN

02

Se mide con **sys.dm_db_index_physical_stats**. Entre 5%-30% se recomienda **REORGANIZE**; por encima de 30%, **REBUILD**.

ESTADÍSTICAS

03

Histogramas que estiman cuántas filas cumplirán un filtro. Se actualizan con **UPDATE STATISTICS** o de forma automática con **AUTO_UPDATE_STATISTICS**.

► SCRIPT T-SQL DE REFERENCIA

```
SELECT index_id, avg_fragmentation_in_percent
FROM sys.dm_db_index_physical_stats(
    DB_ID(), OBJECT_ID(GradosTitulos.dbo.Titulo), NULL, NULL, LIMITED);

ALTER INDEX ALL ON dbo.Titulo REBUILD;
UPDATE STATISTICS dbo.Titulo WITH FULLSCAN;
```

GANANCIA DE RENDIMIENTO

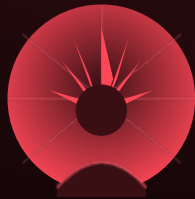
AL RECONSTRUIR
ÍNDICES Y ACTUALIZAR
ESTADÍSTICAS
PERIÓDICAMENTE

80%

BUENAS PRÁCTICAS

- ✓ Programar mantenimiento periódico (Maintenance Plan / scripts tipo Ola Hallengren).
- ✓ Evitar el exceso de índices: cada uno cuesta escritura adicional.
- ✓ Definir un **FILLFACTOR** adecuado para reducir fragmentación futura.
- ✓ Actualizar estadísticas tras cargas masivas de datos.

TEMA 3

CONCURRENCIA Y CONSISTENCIA

ADMINISTRACIÓN DE TRANSACCIONES Y BLOQUEOS

Las transacciones garantizan las propiedades ACID; los bloqueos son el mecanismo que usa SQL Server para que múltiples usuarios accedan a GradosTítulos sin corromper la información.

NIVELES DE AISLAMIENTO

READ COMMITTED (por defecto), **REPEATABLE READ**, **SERIALIZABLE** y **SNAPSHOT** equilibran consistencia contra concurrencia.

01

TIPOS DE BLOQUEO

Compartido (S), exclusivo (X), actualización (U) e intención (IS/IX), aplicados sobre filas, páginas o tablas según el motor lo decida.

02

DEADLOCKS

SQL Server detecta ciclos de espera y elige una **víctima** automáticamente. Se diagnostican con el evento **xml_deadlock_report**.

03

► SCRIPT T-SQL DE REFERENCIA

```
BEGIN TRANSACTION;  
UPDATE dbo.Titulo SET Estado=Aprobado WHERE IdTitulo=105;  
-- ver bloqueos activos de la sesión actual  
SELECT * FROM sys.dm_tran_locks  
WHERE request_session_id = @@SPID;  
COMMIT TRANSACTION;
```

RIESGO DE BLOQUEO EN ALTA CONCURRENCIA

SIN CONTROL DE AISLAMIENTO NI TRANSACCIONES CORTAS

70%

BUENAS PRÁCTICAS

- ✓ Mantener las transacciones lo más **cortas** posible.
- ✓ Acceder a las tablas siempre en el **mismo orden** para evitar deadlocks.
- ✓ Usar **READ_COMMITTED_SNAPSHOT** para reducir bloqueos de lectura.
- ✓ Monitorear bloqueos activos con **sys.dm_tran_locks**.



TEMA 4 OPTIMIZADOR DE CONSULTAS

ANÁLISIS DE PLANES DE EJECUCIÓN

El plan de ejecución revela cómo el optimizador decide acceder y unir los datos. Leerlo correctamente permite ubicar con precisión el origen de una consulta lenta.

ESTIMADO VS. REAL

01

El plan **estimado** usa estadísticas antes de ejecutar; el **real** agrega filas y tiempos reales para contrastarlos y detectar estimaciones erróneas.

OPERADORES COMUNES

02

Index Seek (búsqueda directa) es preferible a **Table/Index Scan** (revisa todo). Las uniones usan Nested Loops, Hash Match o Merge Join.

SEÑALES DE ALERTA

03

Flechas gruesas (muchas filas), advertencias de conversión implícita de tipos y scans sobre tablas grandes indican puntos a optimizar.

► SCRIPT T-SQL DE REFERENCIA

```
SET STATISTICS IO ON;  
SET STATISTICS TIME ON;  
SET SHOWPLAN_XML ON;  
GO  
SELECT IdTitulo, Alumno FROM dbo.Titulo  
WHERE Anio = 2026;
```

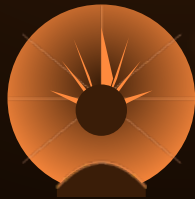
CAPACIDAD DE DETECTAR CUELLOS DE BOTELLA

ANALIZANDO EL PLAN
REAL JUNTO CON
STATISTICS IO/TIME

85%

BUENAS PRÁCTICAS

- ✓ Comparar el **costo relativo** de cada operador dentro del plan.
- ✓ Crear índices que conviertan un Scan en un **Seek**.
- ✓ Revisar siempre **SET STATISTICS IO/TIME** junto al plan gráfico.
- ✓ Guardar planes (.sqlplan) de consultas críticas para comparar en el tiempo.



TEMA 5 ESCRITURA EFICIENTE

OPTIMIZACIÓN DE CONSULTAS T-SQL

La forma en que se escribe una consulta determina si el motor puede aprovechar los índices existentes. Pequeños cambios de sintaxis generan grandes ganancias de rendimiento.

EVITAR SELECT *

Traer solo las columnas necesarias reduce lecturas lógicas, memoria usada y tráfico de red hacia la aplicación.

01

CONSULTAS SARGABLES

Evitar funciones sobre la columna filtrada (ej. **YEAR(Fecha)**) permite que el motor sí utilice el índice disponible.

02

JOINS Y CURSORES

Usar claves indexadas en los JOIN y reemplazar cursores por operaciones basadas en conjuntos reduce drásticamente el tiempo de ejecución.

03

► SCRIPT T-SQL DE REFERENCIA

```
-- No SARGable: el índice de FechaEmision no se usa
SELECT * FROM dbo.Titulo
WHERE YEAR(FechaEmision) = 2026;

-- SARGable: aprovecha el índice
SELECT IdTitulo, Alumno FROM dbo.Titulo
WHERE FechaEmision BETWEEN 2026-01-01 AND 2026-12-31;
```

REDUCCIÓN DEL TIEMPO DE RESPUESTA

AL REESCRIBIR CONSULTAS NO-SARGABLES A SARGABLES

75%

BUENAS PRÁCTICAS

- ✓ Usar **EXISTS** en vez de **IN** con subconsultas grandes.
- ✓ Evitar **cursores**; preferir operaciones basadas en conjuntos.
- ✓ Parametrizar las consultas para reutilizar planes en caché.
- ✓ Evitar conversiones implícitas de tipos en los filtros WHERE.

TEMA 6 GOBIERNO DE RECURSOS



CONTROL DE RECURSOS CON RESOURCE GOVERNOR

Resource Governor limita cuánta CPU, memoria y E/S puede consumir cada tipo de carga de trabajo, evitando que un proceso pesado de reportes afecte las operaciones diarias de GradosTitulos.

POOLS DE RECURSOS

Definen límites de **CPU y memoria** (MIN/MAX) que aplican a un conjunto de sesiones agrupadas.

01

GRUPOS DE CARGA DE TRABAJO

Agrupan sesiones dentro de un pool, aplicando reglas específicas como prioridad o grado de paralelismo.

02

FUNCIÓN DE CLASIFICACIÓN

Función definida por el usuario que asigna cada conexión entrante a un grupo según login, aplicación u horario.

03

► SCRIPT T-SQL DE REFERENCIA

```
CREATE RESOURCE POOL PoolReportes
WITH (MAX_CPU_PERCENT = 40, MAX_MEMORY_PERCENT = 30);
CREATE WORKLOAD GROUP GrupoReportes
USING PoolReportes;
ALTER RESOURCE GOVERNOR RECONFIGURE;
```

CONTROL SOBRE PICOS DE CONSUMO

AISLANDO REPORTES
PESADOS DE LAS
OPERACIONES
TRANSACCIONALES

78%

BUENAS PRÁCTICAS

- ✓ Separar cargas **OLTP** de reportes o cargas masivas pesadas.
- ✓ Monitorear con **sys.dm_resource_governor_resource_pools**.
- ✓ Ajustar los límites de forma progresiva, no agresiva.
- ✓ Documentar la función de clasificación para futuros mantenimientos.